



Bitácora de Desarrollo: Task Manager (Sostecnible)

1. Proceso de Desarrollo

El proyecto se diseñó bajo un enfoque de Arquitectura Desacoplada, permitiendo que el Backend y el Frontend funcionen de manera independiente y escalable.

Front-End (React + Vite)

- **Configuración del Entorno:** Se utilizó Vite para inicializar la SPA por su velocidad en el desarrollo y eficiencia en el manejo de HMR.
- **Arquitectura:** Se implementó una estructura basada en componentes funcionales, centralizando el estado global con Zustand y la lógica de peticiones con React Query.
- **Integración de API:** Se configuró un servicio base utilizando Axios para conectar con el backend a través de la URL definida en las variables de entorno (VITE_URL_API).
- **Ciclo de Pruebas:** Se desarrollaron pruebas unitarias (como taskDetail.test.js) utilizando Vitest y React Testing Library para asegurar el correcto renderizado y la lógica de negocio.

Back-End (Java / Spring Boot)

- **Arquitectura Hexagonal:** Se aplicaron los principios de Clean Architecture para separar el dominio de los frameworks y la persistencia.
- **Modelo de Datos:** Implementación de un CRUD completo para tareas con campos de Título, Descripción obligatoria, Prioridad, Fechas y Estado.



- **Persistencia:** Uso de Spring Data JPA con MySQL para el almacenamiento de la información.
 - **Seguridad y Configuración:** Uso de dotenv-java para gestionar datos sensibles mediante un archivo .env externo.
-

2. Desafíos Técnicos Encontrados

- **Sincronización de Estado y Cache:** Mantener la interfaz actualizada tras operaciones CRUD (Creación, Actualización, Eliminación) sin recargas manuales.
 - **Desacoplamiento de Lógica:** Mantener componentes de React limpios, delegando responsabilidades a Custom Hooks y Stores.
 - **Validaciones Cruzadas:** Asegurar que la regla de negocio "Descripción obligatoria" se cumpla tanto en el cliente (UX) como en el servidor (Integridad de datos).
 - **Arquitectura Hexagonal:** La curva de aprendizaje inicial para separar correctamente Puertos y Adaptadores en el Backend para garantizar un código mantenible.
-

3. Soluciones Implementadas

- **TanStack Query (React Query):** Se utilizó para gestionar el ciclo de vida de los datos. Mediante invalidateQueries, se logró que el listado se actualice automáticamente tras cada mutación.
- **Zustand:** Elegido por su ligereza frente a Redux, facilitando el manejo de filtros por Prioridad y Estado de forma intuitiva.
- **Spring Validation:** Uso de anotaciones en el Backend para validar que la descripción no esté vacía antes de persistir la tarea.



- **Pruebas Unitarias:** Creación de tests específicos para validar que los componentes y servicios procesen la información correctamente antes de la entrega final.

Fecha de finalización: 01 de Marzo de 2026.

Candidato: Yonner Suarez.